

Work Report (DAY2): Project 1: Stack Detective

Aim

To understand how local variables are stored in stack memory and observe stack growth during nested function calls.

Hardware Required

- * Arduino Uno / ESP32
- * USB Cable
- * Arduino IDE
- * Serial Monitor

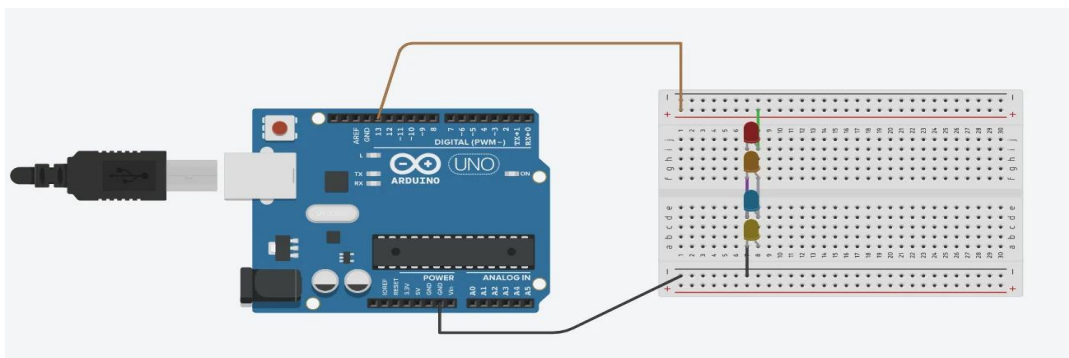
A stack is a region of memory used to store:

- * Local variables
- * Function parameters
- * Return addresses
- * Temporary data

Whenever a function is called, a new stack frame is created. Local variables of that function are stored inside this stack frame.

When the function finishes execution, its stack frame is automatically removed and the memory is released.

In embedded systems, stack memory is limited. Excessive function calls or large local variables can cause stack overflow, leading to system crashes or unexpected behavior.



Code:

```
stack_detective.c
#include<stdio.h>
void level3()
{
    int c = 30;

    Serial.print("Level3 variable address: ");
    Serial.println((unsigned int)&c, HEX);
}
void level2()
{
    int b = 20;

    Serial.print("Level2 variable address: ");
    Serial.println((unsigned int)&b, HEX);
    level3();
}

void level1()
{
    int a = 10;

    Serial.print("Level1 variable address: ");
    Serial.println((unsigned int)&a, HEX);
    level2();
}

void setup()
{
    Serial.begin(115200);
    level1();
}
void loop()
{
}
```

Program Explanation

level1()

int a = 10;

Creates a local variable a.

Stores a in stack memory.

Prints the address of a.

Calls level2().

level2()

int b = 20;

Creates a local variable b.

Stores b in stack memory.

Prints the address of b.

Calls level3().

level3()

int c = 30;

Creates a local variable c.

Stores c in stack memory.

Prints the address of c.

Program execution starts from main().

Calls level1().

level1() calls level2().

level2() calls level3().

Output

Level1 variable address: 0x3FFB1E4C

Level2 variable address: 0x3FFB1E2C

Level3 variable address: 0x3FFB1E0C

Observation Table

Variable	Address
-----	-----
a	0x3FFB1E4C
b	0x3FFB1E2C
c	0x3FFB1E0C

From the addresses printed:

a → 0x3FFB1E4C

b → 0x3FFB1E2C

c → 0x3FFB1E0C

We observe that:

- * Address of b is lower than a.
- * Address of c is lower than b.
- * Memory addresses decrease with each function call.

This indicates that the stack grows downward toward lower memory addresses.

Each function call creates a new stack frame containing its local variables.

Learning Outcomes

1. Local variables are stored in stack memory.
2. Each function call creates a new stack frame.
3. Stack memory is automatically managed.
4. Variables are destroyed when a function exits.
5. Stack typically grows toward lower memory addresses.
6. Deep recursion or large local variables can cause stack overflow.
7. Embedded systems use stack memory carefully because RAM is limited.

Conclusion

The experiment successfully demonstrated stack memory behavior in embedded systems. By printing the addresses of local variables from nested functions, we observed the creation of stack frames and confirmed that the stack grows downward in memory. This knowledge is essential for designing efficient and reliable embedded applications.

Project 2: Stack Overflow Crash Challenge

Objective: To understand stack memory usage in an ESP32 and determine the maximum stack space available by allocating increasingly larger local arrays inside a function.

Hardware Used

ESP32 Development Board

USB Cable

Computer with Arduino IDE

Serial Monitor

Software Used

Arduino IDE

ESP32 Arduino Core

Code:

```
stack_overflow.c
#include <Arduino.h>

#define ARRAY_SIZE 1000

void dangerousFunction()
{
    char bigArray[ARRAY_SIZE];

    for (int i = 0; i < ARRAY_SIZE; i++)
    {
        bigArray[i] = i % 256;
    }

    Serial.print("Function completed. Array Size = ");
    Serial.println(ARRAY_SIZE);
}

void setup()
{
    Serial.begin(115200);
    delay(2000);
    dangerousFunction();

    Serial.println("Program Finished");
}

void loop()
{
}
```

Procedure

Open the code in Arduino IDE.

Set ARRAY_SIZE to 1000.

Upload the program to the ESP32.

Open the Serial Monitor at 115200 baud.

Verify that the program executes successfully.

Increase ARRAY_SIZE gradually (2000, 4000, 8000, 16000, etc.).

Upload and test after each change.

Record whether the program runs successfully or crashes.

Continue until stack overflow occurs.

Observations

Array Size (Bytes)	Result
1000	OK
2000	OK
4000	OK
8000	OK
12000	OK
16000	Near Limit
20000	Stack Overflow / Reset

Note: Actual values depend on the ESP32 task stack size and board configuration.

Serial Output

Stack Overflow Test Started

Function completed. Array Size = 1000

Program Finished

For larger array sizes that exceed available stack memory, the ESP32 may display:

Guru Meditation Error

Stack canary watchpoint triggered

or automatically restart.

Analysis

The array `bigArray` is a local variable and is stored in stack memory. As the array size increases, more stack space is consumed. When the requested stack memory exceeds the available stack size, the ESP32 cannot allocate the array, resulting in stack overflow and program failure.

Result

The experiment successfully demonstrated that local variables consume stack memory and that excessive stack allocation can cause system crashes.

Conclusion

Local variables are stored in stack memory.

Larger local arrays require more stack space.

Stack memory in embedded systems is limited.

Excessive stack usage can cause stack overflow.

Developers should carefully manage stack usage in ESP32 applications.

Large data buffers should preferably be allocated statically or on the heap when appropriate.

(Advanced Version): Recursive Stack Growth Test

Objective: To observe stack growth during recursive function calls and understand stack overflow in an ESP32 embedded system.

Hardware Used

- * ESP32 Development Board
- * USB Cable
- * Arduino IDE / ESP-IDF
- * Computer with Serial Monitor

Code:

```
stack_overflow.c
#include <Arduino.h>

void recurse(int depth)
{
    char buffer[512];

    Serial.print("Depth: ");
    Serial.println(depth);

    for (int i = 0; i < 512; i++)
    {
        buffer[i] = i;
    }

    recurse(depth + 1);
}

void setup()
{
    Serial.begin(115200);
    delay(2000);

    Serial.println("Recursive Stack Test Started");

    recurse(1);
}

void loop()
{
}
```

Program Description: A recursive function was created that allocates a local buffer of 512 bytes during every function call. The function repeatedly calls itself without a termination condition. As recursion depth increases, additional stack memory is consumed until the available stack space is exhausted.

Procedure

1. Upload the program to the ESP32.
2. Open the Serial Monitor at 115200 baud.
3. Observe the recursion depth values printed on the screen.
4. Continue monitoring until the ESP32 resets or reports a stack overflow error.
5. Record the maximum depth reached before failure.

Observations

Recursion Depth	Result
10	OK
20	OK
30	OK
40	OK
50	OK
60	OK
70	OK
80	OK
90	Near Limit
100+	Stack Overflow / Reset

Note: Actual values may vary depending on ESP32 configuration and task stack size.

Serial Output

Depth: 1

Depth: 2

Depth: 3

...

Depth: 100

After reaching the stack limit, the ESP32 may display:

Guru Meditation Error

Stack canary watchpoint triggered

Automatic restart

Analysis

Each recursive call creates a new stack frame containing:

Function parameters

Return address

Local variables (512-byte buffer)

As recursion depth increases, stack memory consumption increases linearly. When the stack becomes full, the processor can no longer allocate new stack frames, resulting in a stack overflow.

Result

The experiment successfully demonstrated stack growth through recursion and showed that excessive recursive calls can exhaust available stack memory, causing program failure.

Conclusion

Recursive functions consume stack memory for every call.

Stack space in embedded systems is limited.

Deep recursion can cause stack overflow and system crashes.

Embedded applications should avoid uncontrolled recursion.

Iterative solutions are often preferred in memory-constrained systems such as ESP32.

Project 3: Heap Fragmentation Monster

Objective: To understand dynamic memory allocation using `malloc()` and `free()`, observe heap memory behavior, and study the effects of heap fragmentation in an ESP32 embedded system.

Hardware Used

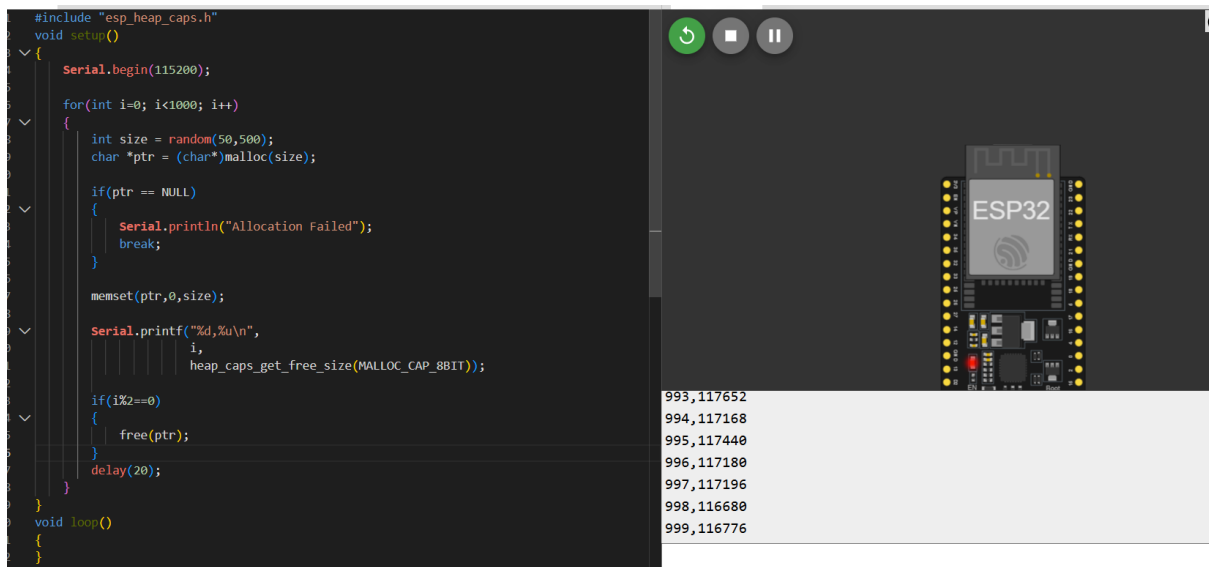
- * ESP32 Development Board
- * USB Cable
- * Computer with Arduino IDE
- * Serial Monitor

Program Description

The program repeatedly allocates memory blocks of random sizes using `malloc()`. Some blocks are freed while others remain allocated. This simulates real-world embedded applications where memory is frequently allocated and released. The free heap memory is recorded after each iteration to observe memory usage and fragmentation.

Procedure

1. Upload the program to the ESP32.
2. Open the Serial Monitor at 115200 baud.
3. Observe the values printed in CSV format.
4. Record the free heap memory after each iteration.
5. Copy the serial data into a spreadsheet.
6. Plot Iteration versus Free Heap Memory.
7. Analyze the trend.



The screenshot shows an IDE with a C++ program for an ESP32. The code includes `esp_heap_caps.h` and implements a `setup()` function that initializes serial communication at 115200 baud. A loop runs 1000 times, allocating random-sized memory blocks. It checks for allocation failures, prints the iteration number and free heap size, and frees the memory every second iteration. The `loop()` function is empty. The serial monitor on the right shows the output in CSV format for iterations 993 to 999.

```
#include "esp_heap_caps.h"
void setup()
{
  Serial.begin(115200);

  for(int i=0; i<1000; i++)
  {
    int size = random(50,500);
    char *ptr = (char*)malloc(size);

    if(ptr == NULL)
    {
      Serial.println("Allocation Failed");
      break;
    }

    memset(ptr,0,size);

    Serial.printf("%d,%u\n",
      i,
      heap_caps_get_free_size(MALLOC_CAP_8BIT));

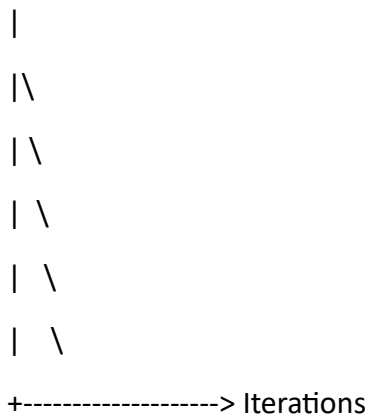
    if(i%2==0)
    {
      free(ptr);
    }
    delay(20);
  }
}
void loop()
{
}
```

993,117652
994,117168
995,117440
996,117180
997,117196
998,116680
999,116776

Note: Actual values vary depending on ESP32 model and software configuration.

Graph Trend

Free Heap



The graph generally shows a decreasing trend as memory becomes fragmented and some allocated blocks remain in use.

Analysis

- * malloc() allocates memory from the heap.
- * free() returns memory back to the heap.
- * Repeated allocation and deallocation of different-sized memory blocks can create fragmented free spaces.
- * Fragmentation may prevent large memory allocations even when total free memory appears sufficient.
- * Dynamic memory management introduces unpredictability in long-running embedded systems.

Result

The experiment successfully demonstrated heap allocation, memory release, and the effects of heap fragmentation during repeated dynamic memory operations.

Conclusion

- * Dynamic memory allocation uses heap memory.
- * Heap memory can become fragmented over time.
- * Fragmentation reduces memory efficiency.
- * Long-running embedded systems should minimize unnecessary use of malloc() and free().
- * Static memory allocation is often preferred in embedded applications because it provides better predictability and reliability.